
Parametric Clustering

K-means, Gaussian mixtures, and expectation–maximization

Kangning Cui

City University of Hong Kong (Dongguan)

Semester A, 2026/27

Learning objectives

Use estimation and Bayes' rule when cluster labels are hidden

1. explain what a clustering result assumes about similarity;
2. derive the assignment and centroid updates in K-means;
3. build a Gaussian mixture model for uncertain cluster membership;
4. use Bayes' rule to calculate soft membership probabilities;
5. derive the two updates in expectation-maximization;
6. diagnose initialization, scaling, and model-selection failures.

Clustering and K-means

CS5487 ■ Semester A, 2026/27

Why do we need clustering?

Suppose we have many observations but no group labels. For example, patients may be represented by feature vectors:

$$\mathbf{x}_i \in \mathbb{R}^d, \quad i = 1, \dots, n.$$

We may want to discover:

- ▶ which patients have similar profiles;
- ▶ whether recurring subgroups exist;
- ▶ whether those groups suggest different patterns or behaviors.

Similar problems appear in customer segmentation, image organization, and document grouping.

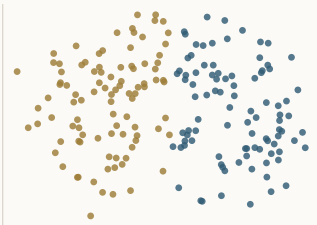
Why is clustering ambiguous?

The same dataset can have different valid clusterings.

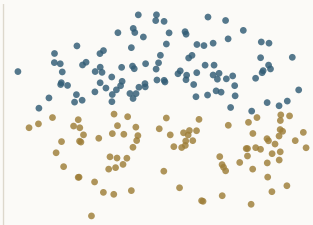
The result depends on:

- ▶ how similarity is defined;
- ▶ what structure the model assumes.

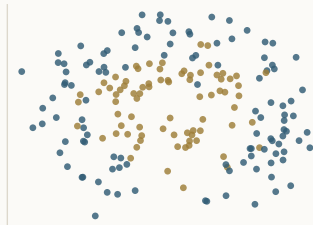
split left / right



split top / bottom



center / outer ring



A clustering result reflects modeling assumptions, not a unique ground truth.

From similarity to K-means

To build a clustering algorithm, we must first decide what “similar” means. K-means makes a geometric choice:

$$\|x_i - \mu_k\|_2^2.$$

Here μ_k is a representative point for cluster k . The assumption is simple:

- ▶ points near the same center are considered similar;
- ▶ compact groups are preferred.

This turns clustering into an optimization problem based on Euclidean distance.

K-means: the basic idea

First choose K , the number of clusters. Represent the clusters using K centroids:

$$\mu_1, \dots, \mu_K.$$

K-means then repeats two steps:

1. assign each point to the closest centroid;
2. move each centroid to the mean of its assigned points.

Therefore the algorithm alternates between assignment and centroid update.

K-means searches for K compact groups around representative centers.

Start with one K-means step in one dimension

Points are 1, 2, 7, 8 and the initial centers are $\mu_1 = 1$, $\mu_2 = 5$.
First compute distances:

point	1	2	7	8
$ x - \mu_1 $	0	1	6	7
$ x - \mu_2 $	4	3	2	3

Therefore:

$$\{1, 2\} \rightarrow 1, \quad \{7, 8\} \rightarrow 2.$$

The first step assigns points according to the nearest centroid.

Updating the centroids

After assignment, replace each center by the mean of its assigned points:

$$\mu_1 = \frac{1 + 2}{2} = 1.5,$$

$$\mu_2 = \frac{7 + 8}{2} = 7.5.$$

The objective decreases because the new centers are closer to their assigned points:

Before update:

$$(1 - 1)^2 + (2 - 1)^2 + (7 - 5)^2 + (8 - 5)^2 = 14.$$

After update:

$$(1 - 1.5)^2 + (2 - 1.5)^2 + (7 - 7.5)^2 + (8 - 7.5)^2 = 1.$$

Generalize the K-means objective

Let $z_i \in \{1, \dots, K\}$ be the assigned cluster and let μ_k be cluster k 's center.
K-means minimizes:

$$J(\mathbf{z}, \boldsymbol{\mu}) = \sum_{i=1}^n \|\mathbf{x}_i - \boldsymbol{\mu}_{z_i}\|_2^2.$$

This objective is the total within-cluster variation.

A good clustering should make:

$$\mathbf{x}_i \approx \boldsymbol{\mu}_{z_i}$$

for points assigned to the same cluster.

Therefore K-means minimizes variation inside clusters rather than directly maximizing separation.

Indicator representation

Instead of storing cluster labels directly, we use indicators.

Define:

$$r_{ik} = 1$$

if point i belongs to cluster k , and:

$$r_{ik} = 0$$

otherwise.

Because each point belongs to exactly one cluster:

$$\sum_{k=1}^K r_{ik} = 1.$$

The indicator r_{ik} converts the discrete assignment into a mathematical form.

K-means objective with indicators

Using indicators, the K-means objective becomes:

$$J = \sum_{i=1}^n \sum_{k=1}^K r_{ik} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2.$$

When:

$$r_{ik} = 1,$$

point i contributes the distance to cluster k .

When:

$$r_{ik} = 0,$$

that cluster does not contribute.

Therefore K-means optimizes two unknown parts:

$$r_{ik}, \text{ and } \boldsymbol{\mu}_k.$$

Why does K-means alternate?

The objective contains two different types of unknown variables:

$$r_{ik}$$

are discrete assignment variables, while:

$$\mu_k$$

are continuous parameters.

Optimizing both simultaneously is difficult.

K-means therefore solves two easier problems:

Fix centers $\rightarrow$ update assignments

and:

Fix assignments $\rightarrow$ update centers.

Each step optimizes one part while keeping the other fixed.

Assignment step: nearest centroid

Fix centroids.

Let z_i denote the cluster label assigned to point i . For each point i , only its own assignment affects the objective:

$$z_i^{\text{new}} = \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2.$$

Therefore each point can be optimized independently.

The indicator update is:

$$r_{ik}^{\text{new}} = 1$$

for the nearest cluster and:

$$r_{ik}^{\text{new}} = 0$$

for all other clusters.

Centroid step: optimization problem

Fix assignments and consider cluster k :

$$J_k(\boldsymbol{\mu}_k) = \sum_i r_{ik} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2.$$

We want the center that minimizes the total squared distance.
Differentiate:

$$\nabla_{\boldsymbol{\mu}_k} J_k = 2 \sum_i r_{ik} (\boldsymbol{\mu}_k - \mathbf{x}_i).$$

At the optimum:

$$\sum_i r_{ik} (\boldsymbol{\mu}_k - \mathbf{x}_i) = 0.$$

The next step is to solve this equation for the optimal center.

Centroid step: derive the mean

From the previous equation:

$$\sum_i r_{ik} \boldsymbol{\mu}_k = \sum_i r_{ik} \mathbf{x}_i.$$

Because $\boldsymbol{\mu}_k$ is the same for all assigned points:

$$\boldsymbol{\mu}_k \sum_i r_{ik} = \sum_i r_{ik} \mathbf{x}_i.$$

Therefore:

$$\boldsymbol{\mu}_k = \frac{\sum_i r_{ik} \mathbf{x}_i}{\sum_i r_{ik}}.$$

The new centroid is the average of points assigned to cluster k .

The K-means algorithm

The complete K-means procedure is:

1. Initialize K cluster centers.
2. Assign each point to the nearest center.
3. Update each center using the assigned mean.
4. Repeat until assignments or the objective stop changing.

The two optimization steps are: **assignment step** and **centroid step**

Each step improves the current solution while keeping the other variables fixed.

Why does K-means converge?

During the assignment step:

J cannot increase because each point chooses its closest center.

During the centroid step:

J cannot increase because each center becomes the optimal mean of its assigned points.

Therefore:

$$J^{(t+1)} \leq J^{(t)}.$$

Because the number of possible hard assignments is finite, eventually the algorithm reaches a fixed point.

Common mistake

A fixed point is only a local optimum, not necessarily the globally best clustering.

Initialization matters

K-means is sensitive to the initial centers.

Different initializations can produce different final solutions:

same data + different initialization $\rightarrow$ different clusters.

The reason is that the K-means objective is not jointly convex in:

$$\{z_i\} \text{ and } \{\mu_k\}.$$

Common approaches include:

- ▶ running K-means multiple times;
- ▶ selecting the solution with the smallest objective;
- ▶ using K-means++ initialization.

Check the next K-means iteration

Using the updated centers:

$$\mu_1 = 1.5, \quad \mu_2 = 7.5.$$

The next assignment gives:

$$\{1, 2\} \rightarrow 1, \quad \{7, 8\} \rightarrow 2.$$

The assignments do not change.

Therefore the centroid update also gives:

$$\mu_1 = 1.5, \quad \mu_2 = 7.5.$$

The algorithm has reached a fixed point.

Check your understanding. Could another initialization lead to a different final clustering?

When K-means fails

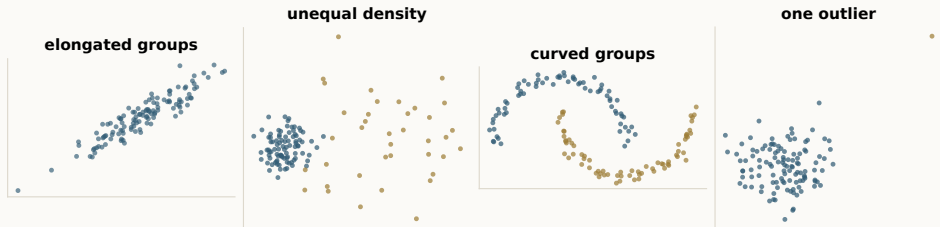
K-means relies on several assumptions:

K-means assumes

- ▶ Clusters are approximately spherical.
- ▶ Euclidean distance represents similarity.
- ▶ The mean is a meaningful cluster center.

Common failures

- ▶ Elongated clusters.
- ▶ Different cluster densities.
- ▶ Non-convex structures.
- ▶ Outliers affecting the mean.



Why these failures happen

Each failure corresponds to a limitation of the K-means model.

- ▶ Elongated clusters: the distance to one center cannot describe the true geometry.
- ▶ Different densities: one centroid does not capture varying cluster scales.
- ▶ Non-convex clusters: Euclidean distance cannot represent complex structures.
- ▶ Outliers: the mean is sensitive to extreme observations.

These limitations motivate probabilistic clustering models.

The weaknesses of K-means reveal the need for more flexible models.

Gaussian mixture models

CS5487 ■ Semester A, 2026/27

Gaussian mixture models

K-means represents each cluster using only a center.

A Gaussian mixture model represents each cluster using:

$$(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \pi_k).$$

The parameters describe:

- ▶ $\boldsymbol{\mu}_k$: cluster location;
- ▶ $\boldsymbol{\Sigma}_k$: cluster shape and uncertainty;
- ▶ π_k : proportion of observations generated by this component.

This allows GMM to represent different cluster shapes and uncertain membership.

Hidden cluster labels

A GMM assumes that each observation is generated by one Gaussian component.
For point i :

$$z_i \in \{1, \dots, K\}$$

indicates the component that generated $\mathbf{x}_i$.

The dataset contains:

$$\mathbf{x}_1, \dots, \mathbf{x}_n,$$

but the labels:

$$z_1, \dots, z_n$$

are unknown.

Therefore GMM must infer hidden assignments from observed data.

How does GMM generate data?

A GMM describes data generation in two steps.

First choose a component:

$$P(z_i = k) = \pi_k.$$

Then generate the observation:

$$\mathbf{x}_i \mid z_i = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Therefore the observed point comes from one of several possible Gaussian distributions.

The cluster identity is hidden, so the model must infer it from the data.

Definition: Gaussian mixture model

A K -component Gaussian mixture model for $\mathbf{x} \in \mathbb{R}^d$ is defined by:

$$\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K.$$

For each component k :

- ▶ $\pi_k \geq 0$ is the mixing weight, with $\sum_{k=1}^K \pi_k = 1$;
- ▶ $\boldsymbol{\mu}_k \in \mathbb{R}^d$ is the mean;
- ▶ $\boldsymbol{\Sigma}_k \in \mathbb{R}^{d \times d}$ is the covariance matrix.

A hidden variable $z \in \{1, \dots, K\}$ selects the component:

$$P(z = k) = \pi_k, \quad \mathbf{x} \mid z = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Marginalizing over the unknown component gives:

$$p(\mathbf{x} \mid \theta) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

A concrete one-dimensional GMM

Consider three observed points:

$$x = \{1, 3, 5\}.$$

Assume a GMM with $K = 2$ components.

For now, suppose the parameters are already known:

$$\pi_1 = \pi_2 = 0.5, \quad \mu_1 = 1, \quad \mu_2 = 5.$$

Both components have variance:

$$\sigma_1^2 = \sigma_2^2 = 1.$$

Because the data are one-dimensional, the covariance matrix reduces to a scalar variance.

Our question is:

which component is more likely to have generated each point?

Step 1: evaluate the Gaussian densities

Start with the middle point:

$$x = 3.$$

For a one-dimensional Gaussian:

$$\mathcal{N}(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

For component 1:

$$\mathcal{N}(3 | 1, 1) \approx 0.0540.$$

For component 2:

$$\mathcal{N}(3 | 5, 1) \approx 0.0540.$$

The point is equally plausible under the two Gaussian components.

Step 2: compute the mixture density

The Gaussian densities alone do not include how common each component is. Multiply each density by its mixing weight:

$$s_1 = \pi_1 \mathcal{N}(3 | 1, 1) = 0.5(0.0540) = 0.0270,$$

$$s_2 = \pi_2 \mathcal{N}(3 | 5, 1) = 0.5(0.0540) = 0.0270.$$

The mixture density is the sum:

$$p(x = 3 | \theta) = s_1 + s_2 = 0.0540.$$

Each s_k measures how strongly component k explains the observation before normalization.

Step 3: turn scores into responsibilities

We now ask which component generated $x = 3$.

Normalize the two component scores:

$$\gamma_{31} = P(z = 1 \mid x = 3, \theta) = \frac{0.0270}{0.0540} = 0.5,$$

$$\gamma_{32} = P(z = 2 \mid x = 3, \theta) = \frac{0.0270}{0.0540} = 0.5.$$

Therefore:

$$(\gamma_{31}, \gamma_{32}) = (0.5, 0.5).$$

The model does not need to force a hard decision.

What happens for a clear point?

Now consider:

$$x = 1.$$

The Gaussian densities are:

$$\mathcal{N}(1 \mid 1, 1) \approx 0.399, \quad \mathcal{N}(1 \mid 5, 1) \approx 0.000134.$$

Including the mixing weights:

$$s_1 = 0.5(0.399) \approx 0.1995,$$

$$s_2 = 0.5(0.000134) \approx 0.000067.$$

The total mixture density is:

$$p(x = 1 \mid \theta) \approx 0.1996.$$

Component 1 explains this point much better.

Responsibility for the clear point

Normalize the two scores:

$$P(z = 1 \mid x = 1, \theta) = \frac{0.1995}{0.1996} \approx 0.9997,$$

$$P(z = 2 \mid x = 1, \theta) = \frac{0.000067}{0.1996} \approx 0.0003.$$

Therefore the model is almost certain that $x = 1$ came from component 1.

By symmetry, for $x = 5$:

$$P(z = 1 \mid x = 5, \theta) \approx 0.0003, \quad P(z = 2 \mid x = 5, \theta) \approx 0.9997.$$

GMM gives confident assignments far from the boundary and uncertain assignments near overlap.

Putting the three points together

For the same GMM:

x	$P(z = 1 \mid x, \theta)$	$P(z = 2 \mid x, \theta)$
1	0.9997	0.0003
3	0.5000	0.5000
5	0.0003	0.9997

The model therefore distinguishes between:

- ▶ clear membership: $x = 1$ and $x = 5$;
- ▶ uncertain membership: $x = 3$.

With K-means centers at 1 and 5, the point $x = 3$ is exactly between the two centers. GMM retains this ambiguity instead of forcing a hard assignment.

But where do the GMM parameters come from?

In the previous example, we were given:

$$\pi_k, \quad \mu_k, \quad \sigma_k^2.$$

With known parameters, computing responsibilities is straightforward.

In real clustering problems, however, we observe only:

$$x_1, \dots, x_n.$$

Both the parameters and hidden memberships are unknown.

To estimate memberships, we need the parameters:

$$\theta \rightarrow \gamma.$$

But to estimate the parameters, we need the memberships:

$$\gamma \rightarrow \theta.$$

Expectation–maximization

CS5487 ■ Semester A, 2026/27

Why do we need EM?

GMM has a circular estimation problem. If the hidden memberships were known, we could estimate:

$$\pi_k, \quad \boldsymbol{\mu}_k, \quad \boldsymbol{\Sigma}_k.$$

If the parameters were known, we could compute:

$$\gamma_{ik} = P(z_i = k \mid \mathbf{x}_i, \theta).$$

But in reality, neither is known. EM resolves this by alternating:

- ▶ estimate memberships using current parameters;
- ▶ update parameters using current memberships.

The EM intuition

EM alternates between two easier problems.

E-step

Assume current parameters are correct.

Estimate hidden cluster memberships:

$$\gamma_{ik} = P(z_i = k | \mathbf{x}_i, \theta).$$

M-step

Assume memberships are known.

Update the model parameters using these probabilities.

The cycle is:

$$\theta^{old} \rightarrow \gamma \rightarrow \theta^{new}.$$

The EM iteration visually

EM alternates two steps:

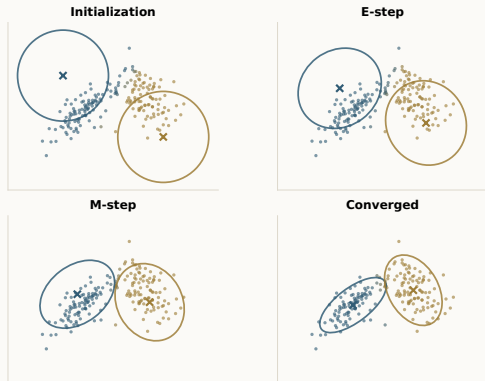
E-step

Infer hidden memberships using current parameters.

M-step

Update model parameters using these memberships.

The cycle repeats until convergence.



E-step: what exactly do we compute?

At iteration t , assume the current GMM parameters are:

$$\theta^{(t)} = \{\pi_k^{(t)}, \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)}\}_{k=1}^K.$$

For each point $\mathbf{x}_i$, the E-step computes the posterior probability that component k generated it:

$$\gamma_{ik}^{(t)} = P(z_i = k \mid \mathbf{x}_i, \theta^{(t)}).$$

Using Bayes' rule:

$$\gamma_{ik}^{(t)} = \frac{\pi_k^{(t)} \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}{\sum_{j=1}^K \pi_j^{(t)} \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})}.$$

For each point:

$$\sum_{k=1}^K \gamma_{ik}^{(t)} = 1.$$

M-step: what exactly do we optimize?

After the E-step, treat the responsibilities $\gamma_{ik}^{(t)}$ as fixed weights.

The M-step chooses new parameters that best fit these weighted assignments.

For a GMM, the objective has the form:

$$Q(\theta) = \sum_{i=1}^n \sum_{k=1}^K \gamma_{ik}^{(t)} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)] .$$

Each observation contributes to every component, but with different weight:

$$\gamma_{ik}^{(t)} .$$

Therefore the M-step becomes weighted maximum-likelihood estimation.

M-step: effective component size

For component k , define:

$$N_k = \sum_{i=1}^n \gamma_{ik}.$$

Why is N_k useful?

With hard labels, we would count how many observations belong to cluster k .

With soft labels, each point contributes only a fraction:

$$0 \leq \gamma_{ik} \leq 1.$$

Therefore N_k is the **effective number of observations** assigned to component k .

Because:

$$\sum_{k=1}^K N_k = n,$$

the updated mixing weight is:

$$\pi_k^{new} = \frac{N_k}{n}.$$

M-step: why is the mean a weighted average?

Fix the responsibilities and covariance of component k .

The part of the objective involving μ_k is equivalent to minimizing:

$$\sum_{i=1}^n \gamma_{ik} \|\mathbf{x}_i - \mu_k\|^2.$$

Differentiate with respect to μ_k :

$$\sum_{i=1}^n \gamma_{ik} (\mu_k - \mathbf{x}_i) = 0.$$

Rearranging:

$$\mu_k \sum_{i=1}^n \gamma_{ik} = \sum_{i=1}^n \gamma_{ik} \mathbf{x}_i.$$

Therefore:

$$\mu_k^{new} = \frac{\sum_i \gamma_{ik} \mathbf{x}_i}{N_k}.$$

M-step: why update the covariance this way?

The mean describes the center of component k .

The covariance should describe how points spread around that center.

With soft assignments, each squared deviation is weighted by its responsibility:

$$\gamma_{ik}(\mathbf{x}_i - \boldsymbol{\mu}_k^{new})(\mathbf{x}_i - \boldsymbol{\mu}_k^{new})^\top.$$

Averaging these weighted deviations gives:

$$\boldsymbol{\Sigma}_k^{new} = \frac{1}{N_k} \sum_{i=1}^n \gamma_{ik}(\mathbf{x}_i - \boldsymbol{\mu}_k^{new})(\mathbf{x}_i - \boldsymbol{\mu}_k^{new})^\top.$$

In one dimension, this becomes:

$$(\sigma_k^2)^{new} = \frac{1}{N_k} \sum_i \gamma_{ik}(x_i - \mu_k^{new})^2.$$

So the three M-step updates have a simple interpretation:

soft count $\rightarrow$ location $\rightarrow$ spread.

A numerical EM example

Continue with the previous one-dimensional dataset:

$$x_1 = 1, \quad x_2 = 3, \quad x_3 = 5.$$

Assume two Gaussian components with initial parameters:

$$\pi_1^{(0)} = \pi_2^{(0)} = 0.5,$$

$$\mu_1^{(0)} = 1, \quad \mu_2^{(0)} = 5,$$

$$(\sigma_1^2)^{(0)} = (\sigma_2^2)^{(0)} = 1.$$

These are only starting guesses.

One EM iteration will perform:

E-step $\rightarrow$ M-step.

E-step: estimate hidden memberships

Using the current parameters $\theta^{(0)}$, compute:

$$\gamma_{ik}^{(0)} = P(z_i = k \mid x_i, \theta^{(0)}).$$

From the previous GMM calculation:

x_i	$\gamma_{i1}^{(0)}$	$\gamma_{i2}^{(0)}$
1	0.9997	0.0003
3	0.5000	0.5000
5	0.0003	0.9997

The interpretation is immediate:

- ▶ $x_1 = 1$ strongly supports component 1;
- ▶ $x_2 = 3$ is shared equally;
- ▶ $x_3 = 5$ strongly supports component 2.

These probabilities now act as weights in the M-step.

M-step: update component sizes

First compute the effective number of observations assigned to each component:

$$N_1 = 0.9997 + 0.5000 + 0.0003 = 1.5,$$

$$N_2 = 0.0003 + 0.5000 + 0.9997 = 1.5.$$

Since there are $n = 3$ observations, update the mixing weights:

$$\pi_1^{(1)} = \frac{N_1}{n} = \frac{1.5}{3} = 0.5,$$

$$\pi_2^{(1)} = \frac{N_2}{n} = 0.5.$$

The mixing weights remain equal because both components receive the same total responsibility.

M-step: update the means

For component 1:

$$\mu_1^{(1)} = \frac{0.9997(1) + 0.5000(3) + 0.0003(5)}{1.5} \approx 1.668.$$

For component 2:

$$\mu_2^{(1)} = \frac{0.0003(1) + 0.5000(3) + 0.9997(5)}{1.5} \approx 4.332.$$

The means move from:

$$(1, 5) \rightarrow (1.668, 4.332).$$

Why?

The uncertain middle point $x = 3$ contributes half of its weight to each component.

The EM mean update is a soft version of the K-means centroid update.

M-step: update the variances

For component 1:

$$(\sigma_1^2)^{(1)} = \frac{\sum_i \gamma_{i1}^{(0)} (x_i - \mu_1^{(1)})^2}{N_1}.$$

Substituting the values:

$$(\sigma_1^2)^{(1)} \approx 0.891.$$

By symmetry:

$$(\sigma_2^2)^{(1)} \approx 0.891.$$

The updated parameter set is therefore:

$$\theta^{(1)} = \{ \pi_1 = \pi_2 = 0.5, \mu_1 = 1.668, \mu_2 = 4.332, \sigma_1^2 = \sigma_2^2 = 0.891 \}.$$

One complete EM iteration is finished.

Start the next EM iteration

EM does not stop after one E-step and one M-step.

Use the updated parameters:

$$\theta^{(1)}$$

to compute new responsibilities.

For the three observations:

x_i	$\gamma_{i1}^{(1)}$	$\gamma_{i2}^{(1)}$
1	0.9975	0.0025
3	0.5000	0.5000
5	0.0025	0.9975

These new responsibilities would be used in the next M-step.

Therefore EM repeats:

$$\theta^{(0)} \rightarrow \gamma^{(0)} \rightarrow \theta^{(1)} \rightarrow \gamma^{(1)} \rightarrow \theta^{(2)} \rightarrow \dots$$

until the parameters and likelihood stabilize.

Why not maximize the GMM likelihood directly?

The likelihood is:

$$\ell(\theta) = \sum_i \log \left(\sum_k \pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right).$$

The problem is:

$\log(\text{sum of components}).$

The logarithm couples all components together.

Therefore we cannot estimate each Gaussian independently.

EM introduces a way to separate the hidden assignments from parameter updates.

Why introduce a lower bound?

The quantity we really want to maximize is:

$$\log p(\mathbf{X} \mid \theta) = \log \sum_z p(\mathbf{X}, z \mid \theta).$$

The difficulty is the log of a sum over all possible hidden assignments. Introduce a probability distribution $q(z)$ over these hidden labels:

$$\sum_z q(z) = 1.$$

Then we can rewrite:

$$\log p(\mathbf{X} \mid \theta) = \log \sum_z q(z) \frac{p(\mathbf{X}, z \mid \theta)}{q(z)}.$$

This is:

$$\log E_q \left[\frac{p(\mathbf{X}, z \mid \theta)}{q(z)} \right].$$

From the likelihood to a lower bound

Because $\log(\cdot)$ is concave, Jensen's inequality gives:

$$\log E_q[Y] \geq E_q[\log Y].$$

Therefore:

$$\log p(\mathbf{X} \mid \theta) \geq \sum_z q(\mathbf{z}) \log \frac{p(\mathbf{X}, \mathbf{z} \mid \theta)}{q(\mathbf{z})}.$$

Define the right-hand side as:

$$\mathcal{F}(q, \theta) = \sum_z q(\mathbf{z}) \log \frac{p(\mathbf{X}, \mathbf{z} \mid \theta)}{q(\mathbf{z})}.$$

Hence:

$$\boxed{\mathcal{F}(q, \theta) \leq \log p(\mathbf{X} \mid \theta)}$$

It is called a lower bound because it always lies below the true log-likelihood.

How does EM use the lower bound?

The gap between the true log-likelihood and the lower bound is:

$$\log p(\mathbf{X} \mid \theta) - \mathcal{F}(q, \theta) = D_{\text{KL}}(q(\mathbf{z}) \parallel p(\mathbf{z} \mid \mathbf{X}, \theta)).$$

Because KL divergence is non-negative, the lower bound can touch the true likelihood only when:

$$q(\mathbf{z}) = p(\mathbf{z} \mid \mathbf{X}, \theta).$$

This explains the two EM steps:

E-step: make the bound tight

Set q equal to the posterior distribution of the hidden labels. For GMM:

$$q(z_i = k) = \gamma_{ik}.$$

M-step: raise the bound

Keep q fixed and update θ to increase $\mathcal{F}(q, \theta)$.

Why does the E-step choose posterior probabilities?

The gap between likelihood and the lower bound is:

$$\log p(\mathbf{X}|\theta) - \mathcal{F}(q, \theta) = D_{\text{KL}}(q(z)||p(z|\mathbf{X}, \theta)).$$

Because KL divergence is non-negative:

$$\mathcal{F}(q, \theta) \leq \log p(\mathbf{X}|\theta).$$

The bound becomes tight when:

$$q(z) = p(z|\mathbf{X}, \theta).$$

Therefore the E-step computes the posterior membership probabilities.

Summary: the EM cycle

The key idea of EM is alternating between two problems:

E-step

Use current parameters to estimate hidden memberships:

$$\gamma_{ik} = P(z_i = k | \mathbf{x}_i, \theta).$$

M-step

Use these memberships to update:

$$\pi_k, \quad \boldsymbol{\mu}_k, \quad \boldsymbol{\Sigma}_k.$$

The algorithm repeats:

$$\theta^{old} \rightarrow \gamma \rightarrow \theta^{new}.$$

Modeling choices and failures

CS5487 ■ Semester A, 2026/27

Why do we need to choose a covariance model?

In K-means, each cluster is represented only by a center.

In GMM, each component also has a covariance matrix:

$$\Sigma_k.$$

The covariance determines what shapes the component is allowed to have.

For example, should a cluster be:

- ▶ roughly circular?
- ▶ stretched differently along different features?
- ▶ rotated because features are correlated?

Different answers correspond to different assumptions about Σ_k .

What does the covariance matrix describe?

Consider a two-dimensional component:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \Sigma_k = \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{bmatrix}.$$

The diagonal entries:

$$\sigma_1^2, \quad \sigma_2^2$$

describe how much the component spreads along each feature.

The off-diagonal term:

$$\sigma_{12}$$

describes whether the two features vary together.

Therefore Σ_k controls:

- ▶ spread;
- ▶ direction;
- ▶ correlation.

What does “covariance structure” mean?

A covariance structure is simply a restriction placed on:

$$\Sigma_k.$$

Different restrictions produce different cluster geometries.

Spherical

$$\Sigma_k = \sigma_k^2 \mathbf{I}$$

Same spread in every direction.

Diagonal

$$\Sigma_k = \text{diag}(\sigma_{k1}^2, \dots, \sigma_{kd}^2)$$

Different spread for each feature, but no feature correlation.

More flexible covariance choices

Two more common choices are:

Full covariance

$$\Sigma_k = \begin{bmatrix} \sigma_{11} & \cdots & \sigma_{1d} \\ \vdots & \ddots & \vdots \\ \sigma_{d1} & \cdots & \sigma_{dd} \end{bmatrix}$$

Variances and correlations are all learned. This allows rotated ellipsoidal clusters.

Tied covariance

All components share one covariance matrix:

$$\Sigma_1 = \Sigma_2 = \cdots = \Sigma_K.$$

The component centers may differ, but their shape and spread are shared.

Flexibility versus overfitting

Increasing model complexity means:

more parameters, and better ability to fit data.

However, with limited data, too many parameters may produce unstable estimates.

For GMM:

complex covariance + small dataset

can lead to unreliable clusters.

A failure case: covariance collapse

A Gaussian component can collapse onto a single observation.

Suppose one point is $x_i = 4$ and component k moves its mean onto this point:

$$\mu_k = 4.$$

For a one-dimensional Gaussian evaluated at its mean:

$$\mathcal{N}(4 | 4, \sigma_k^2) = \frac{1}{\sqrt{2\pi\sigma_k^2}}.$$

As the variance becomes smaller:

$$\sigma_k^2 = 1 \Rightarrow \mathcal{N} \approx 0.399, \quad \sigma_k^2 = 0.01 \Rightarrow \mathcal{N} \approx 3.99.$$

Therefore:

$$\sigma_k^2 \rightarrow 0 \quad \Rightarrow \quad \mathcal{N}(4 | 4, \sigma_k^2) \rightarrow \infty.$$

The likelihood can increase simply because one component becomes extremely narrow around one data point. The model is memorizing one observation rather than discovering a meaningful cluster.

Preventing covariance collapse

Covariance collapse occurs because maximum likelihood allows:

$$\Sigma_k \rightarrow 0.$$

A common solution is to prevent the covariance from becoming arbitrarily small:

$$\Sigma_k \leftarrow \Sigma_k + \varepsilon \mathbf{I}, \quad \varepsilon > 0.$$

This is called **covariance regularization**.

Other practical safeguards include:

- ▶ requiring a minimum effective component size;
- ▶ using multiple EM initializations;
- ▶ checking performance on held-out data.

The key lesson is:

higher training likelihood $\nRightarrow$ better clustering.

How to avoid unreliable mixtures

Practical solutions include:

- ▶ covariance regularization;
- ▶ multiple initializations;
- ▶ minimum component size;
- ▶ validation on unseen data.

The goal is not only:

high likelihood,

but:

stable and interpretable clusters.

How many Gaussian components?

The number of components K is usually unknown. Increasing K improves flexibility, but also increases the risk of overfitting. Common selection criteria include:

- ▶ validation likelihood;
- ▶ Bayesian information criterion (BIC);
- ▶ clustering stability;
- ▶ downstream task performance.

BIC balances fit and model complexity:

$$\text{BIC} = -2\ell(\hat{\theta}) + m \log n,$$

where m is the number of fitted parameters and n is the number of observations. Lower BIC is preferred when comparing models fitted to the same dataset.

A component is not always a real class

A GMM component represents a statistical distribution.
It does not automatically represent:

a meaningful real-world category.

For example:

- ▶ one real group may require several components;
- ▶ one component may only describe a statistical pattern.

Therefore interpretation requires domain knowledge.

Clusters discovered by GMM are model-dependent, not automatically true categories.

Practical GMM implementation

Important choices in practice:

- ▶ number of components;
- ▶ covariance type;
- ▶ initialization;
- ▶ regularization.

Example:

```
from sklearn.mixture import GaussianMixture

model = GaussianMixture(
    n_components=3,
    covariance_type="full",
    reg_covar=1e-6,
    n_init=10
)

model.fit(X_train)
score = model.score(X_test)
```

The implementation choices correspond directly to the modeling assumptions.

Modern applications

CS5487 ■ Semester A, 2026/27

Where are K-means and GMM still useful?

Modern datasets are often high-dimensional, but the basic clustering question remains:
which observations show similar structure?

K-means and GMM are commonly used for:

- ▶ clustering learned feature embeddings;
- ▶ discovering groups when labels are unavailable;
- ▶ generating pseudo-labels for later learning;
- ▶ exploratory analysis and visualization;
- ▶ providing simple and interpretable baselines.

In many modern systems, the pipeline is:

raw data $\rightarrow$ feature representation $\rightarrow$ K-means or GMM.

The clustering model may be classical, while the features being clustered can come from modern representation learning.

Example: hyperspectral image clustering

A hyperspectral image records a spectrum at every pixel.

Instead of only RGB values, each pixel contains many spectral bands:

$$\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{iB}]^T \in \mathbb{R}^B,$$

where B is the number of spectral bands.

Different materials can produce different spectral signatures.

Without pixel-level labels, we may ask: which pixels have similar spectral characteristics?

Clustering can therefore group pixels into candidate material or land-cover regions.

K-means and GMM for hyperspectral pixels

Suppose each pixel is represented by its spectral vector:

$$\mathbf{x}_i \in \mathbb{R}^B.$$

K-means

Pixels are assigned to the nearest spectral centroid:

$$z_i = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2.$$

Output:

- ▶ one hard cluster per pixel;
- ▶ one representative spectrum per cluster.

GMM

Each cluster is modeled probabilistically:

$$P(z_i = k \mid \mathbf{x}_i).$$

Output:

- ▶ soft membership probabilities;
- ▶ component-specific means and covariance.

Why is real hyperspectral clustering harder?

Directly clustering raw spectra is only a starting point.
Real hyperspectral data introduce additional challenges:

- ▶ hundreds of correlated spectral bands;
- ▶ noise and redundant features;
- ▶ spectral variability within the same material;
- ▶ similar spectra from different materials;
- ▶ neighboring pixels also contain spatial information.

A modern pipeline may therefore use:

hyperspectral cube $\rightarrow$ feature extraction / dimensionality reduction $\rightarrow$ clustering.

K-means or GMM can then operate on the learned or reduced representation.

Hidden assignments turn clustering into probabilistic learning

Questions and discussion